AN EXACT CONE QUOTIENT TRANSDUCER FOR GUE SPACINGS

RÔTKIV AYNÄSAC

Abstract. We isolate an exact algebraic mechanism behind a cone model for unfolded nearest-neighbour gaps. The central object is the quotient

$$Q_N = \frac{Y_N - 1}{Y_{N-1}}, \qquad h_N = -\frac{1}{\alpha}\operatorname{Arg}_{\mathrm{lift}}(Q_N), \qquad \nu_N = \frac{h_N}{h_N^{(0)}}.$$

The main theorem states that this map is an exact transducer: for any prescribed spacing sequence s_N, if $h_N = h_N^{(0)} s_N$ lies in the chosen branch and Y_N is generated by

$$Y_N = 1 + e^{-i\alpha h_N} Y_{N-1},$$

then the cone quotient recovers $\nu_N = s_N$ identically. Consequently, if the input sequence has the Wigner–GUE nearest-neighbour law, the cone quotient output has the same law exactly, at finite N, independently of fitted auxiliary coordinates.

For zeta zeros, the relation to the quotient is elementary. If

$$S_N = \sum_{n \le N} e^{i\alpha\gamma_n}, \qquad Y_N = e^{-i\alpha\gamma_N}(S_N - K),$$

then $Y_N - e^{-i\alpha(\gamma_N - \gamma_{N-1})} Y_{N-1} = 1$. Thus the cone quotient recovers the actual zero gaps by telescoping, not by fitting. The numerical section verifies this identity on 2,001,052 Odlyzko zero ordinates stored in `zeross.bin`, generated from Odlyzko's `zeros6.txt` data, and compares the recovered spacings with the Wigner–GUE surmise.

Contents

1. Scope of the Claim 1
2. Constants and Normalization 2
3. The Exact Cone Quotient 2
4. Elementary Relation to Zeta Zeros 3
5. The Reduced G-Law 4
6. Numerical Verification 4
6.1. Data and method 4
6.2. Cone geometry 4
6.3. Exact recovery of gaps 5
6.4. Summary statistics 5
6.5. No-zero transducer test 5
7. Interpretation 5
8. Conclusion 6
Appendix A. Python Code Used for All Figures and Tables 11

1. Scope of the Claim

This note separates two statements which are often conflated.

Date: June 14, 2026.

(1) The cone quotient is an exact algebraic transducer. If a spacing law is inserted into the recurrence for Y_N, then the quotient returns that law exactly.
(2) The Odlyzko zero data exhibit the familiar GUE nearest-neighbour statistics. The cone quotient recovers those same gaps from the elementary walk identity, with no additional fit in the gap extraction step.

The first statement is a theorem. The second is a numerical verification of the Montgomery–Odlyzko phenomenon in the present cone coordinates. The article does not claim to prove the Montgomery–Odlyzko conjecture from first principles. It proves that, once the zeta-zero sequence is related to Y_N, the cone quotient cannot change the spacing law: it is algebraically locked to the gaps.

2. Constants and Normalization

The cone constants used throughout are

$$L = 3^{\pi}\sqrt{e^{\pi}} = 151.74304880712214\ldots, \tag{2.1}$$

$$\alpha = \frac{2\pi}{L} = 0.0414067422301896\ldots, \tag{2.2}$$

$$K = -5.161990134502993 + 2.9872973089668946\,i. \tag{2.3}$$

The constant K locates the cone in the complex plane. It cancels from the exact quotient identity below.

For a height sequence $H_N > 2\pi$, define the smooth local mean gap

$$h_N^{(0)} = \frac{2\pi}{\log(H_N/(2\pi))}. \tag{2.4}$$

In numerical experiments without real zeros, H_N is the inverse of the smooth Riemann–von Mangoldt equation

$$F(T) = \frac{T}{2\pi}\log\frac{T}{2\pi} - \frac{T}{2\pi} + \frac{7}{8}, \qquad F(H_N) = N. \tag{2.5}$$

This is an exact definition of the smooth height used in the experiment; it is not a claim that $H_N = \gamma_N$.

The smooth cone amplitude is

$$A_N = \frac{1}{2} + \frac{1}{2\pi\alpha^2 H_N} - \frac{i}{2\pi\alpha}\log\frac{H_N}{2\pi}. \tag{2.6}$$

When real zeros are used, $H_N = \gamma_N$. When no real zeros are used, H_N is the smooth RVM inverse height.

3. The Exact Cone Quotient

Definition 3.1. Let $Y_N \neq 0$ be a complex sequence and let $H_N > 2\pi$. Define

$$Q_N = \frac{Y_N - 1}{Y_{N-1}}, \qquad h_N = -\frac{1}{\alpha}\operatorname{Arg}_{\mathrm{lift}}(Q_N), \qquad \nu_N = \frac{h_N}{h_N^{(0)}}. \tag{3.1}$$

The lifted argument is taken on the branch used by the implemented function: after reducing modulo $2\pi/\alpha$, the local positive branch is selected.

Theorem 3.2 (Exact transducer theorem)**.** *Fix $\alpha > 0$, a height sequence $H_N > 2\pi$, and a prescribed sequence $s_N > 0$. Put*

$$h_N = h_N^{(0)} s_N.$$

Assume h_N lies in the selected argument branch, and define

$$Y_N = 1 + e^{-i\alpha h_N} Y_{N-1}$$

from any nonzero initial value Y_1. Then the cone quotient applied to this Y returns

$$\nu_N = s_N$$

for every N for which the recurrence is defined.

Proof. The recurrence gives

$$Y_N - 1 = e^{-i\alpha h_N} Y_{N-1}.$$

Since $Y_{N-1} \neq 0$,

$$Q_N = \frac{Y_N - 1}{Y_{N-1}} = e^{-i\alpha h_N}.$$

Taking the lifted branch of the argument gives

$$-\frac{1}{\alpha} \operatorname{Arg}_{\text{lift}}(Q_N) = h_N.$$

Therefore

$$\nu_N = \frac{h_N}{h_N^{(0)}} = s_N.$$

□

Corollary 3.3 (Distribution preservation)**.** *Let μ be any target spacing distribution supported inside the selected branch. If the finite empirical measure of $s_1, \ldots, s_M$ is μ_M, then the finite empirical measure of the cone output $\nu_1, \ldots, \nu_M$ is exactly μ_M.*

Proof. Theorem 3.2 gives equality term by term: $\nu_N = s_N$. Thus every empirical statistic of the output sequence equals the corresponding empirical statistic of the input sequence. □

Corollary 3.4 (Wigner–GUE input gives Wigner–GUE output)**.** *For the Wigner–GUE nearest-neighbour surmise*

$$p_{\text{GUE}}(s) = \frac{32}{\pi^2} s^2 e^{-4s^2/\pi},$$

any sequence s_N whose empirical law is this law is returned unchanged by the cone quotient, after the recurrence construction above. The CDF used in the computations is

$$F_{\text{GUE}}(s) = \operatorname{erf}\left(\frac{2s}{\sqrt{\pi}}\right) - \frac{4s}{\pi} e^{-4s^2/\pi}.$$

4. Elementary Relation to Zeta Zeros

Let γ_N be the positive ordinates of the nontrivial zeros of the Riemann zeta function, ordered increasingly, and define

$$R_N = e^{i\alpha\gamma_N}, \qquad S_N = \sum_{n=1}^{N} R_n. \tag{4.1}$$

For any fixed complex constant K, put

$$Y_N = R_N^{-1}(S_N - K) = e^{-i\alpha\gamma_N}(S_N - K). \tag{4.2}$$

Theorem 4.1 (Telescoping identity for zeros)**.** *Let $h_N = \gamma_N - \gamma_{N-1}$. Then*

$$Y_N - e^{-i\alpha h_N} Y_{N-1} = 1.$$

Consequently,

$$\frac{Y_N - 1}{Y_{N-1}} = e^{-i\alpha h_N},$$

and the cone quotient recovers the actual zero gap h_N on the selected branch.

Proof. Since $h_N = \gamma_N - \gamma_{N-1}$,

$$e^{-i\alpha h_N} R_{N-1}^{-1} = R_N^{-1}.$$

Therefore

$$\begin{aligned} Y_N - e^{-i\alpha h_N} Y_{N-1} &= R_N^{-1}(S_N - K) - e^{-i\alpha h_N} R_{N-1}^{-1}(S_{N-1} - K) \\ &= R_N^{-1}(S_N - K) - R_N^{-1}(S_{N-1} - K) \\ &= R_N^{-1}(S_N - S_{N-1}) \\ &= R_N^{-1} R_N \\ &= 1. \end{aligned}$$

The quotient identity follows immediately. □

Remark 4.2. The proof shows why fitted coordinates are accidental for the extraction of the gaps. The constant K cancels. The quotient uses only the algebraic relation between S_N, S_{N-1}, and R_N.

5. The Reduced G-Law

Writing

$$Y_N = A_N + G_N$$

gives the exact quotient

$$Q_N = \frac{A_N + G_N - 1}{A_{N-1} + G_{N-1}}.$$

The numerical observation in the present cone is that the local fluctuation is carried almost entirely by the real increment of G_N:

$$\nu_N \approx 1 - \Re(G_N - G_{N-1}). \tag{5.1}$$

For the Odlyzko data used below, the correlation between the two sides is

$$0.99999380760483703,$$

with RMS difference

$$1.4414149438073151 \cdot 10^{-3}.$$

This is not needed for Theorem 3.2; it explains why a natural model of G_N-increments is enough to generate the observed spacing law.

6. Numerical Verification

6.1. **Data and method.** The computations were done in Python from the file `zeross.bin`. The file contains 2,001,052 ordinates, generated from Odlyzko's `zeros6.txt` data. The first and last ordinates used here are

$$\gamma_1 = 14.134725142000001, \qquad \gamma_N = 1132490.6587144111.$$

All code used to generate the figures and tables is reproduced in Appendix A.

6.2. **Cone geometry.** Figure 1 shows the projected three-dimensional walk $(\Re S_N, \Im S_N, \gamma_N)$. Figure 2 overlays the walk with the smooth cone term $K + R_N A_N$. The purpose of the second figure is not to assert that the smooth term alone predicts every zero. It shows the geometric coordinate system in which the exact quotient identity is evaluated.

6.3. **Exact recovery of gaps.** The direct unfolded gaps are

$$\nu_N^{\mathrm{direct}} = \frac{\gamma_N - \gamma_{N-1}}{2\pi/\log(\gamma_N/(2\pi))}.$$

The cone quotient gaps are computed from $Y_N = e^{-i\alpha\gamma_N}(S_N - K)$. The maximum absolute error in the recovered raw gaps is

$$\max_N |h_N^{\mathrm{cone}} - h_N^{\mathrm{direct}}| = 1.755231515687683 \cdot 10^{-10},$$

and the RMS error is

$$4.679744130769421 \cdot 10^{-11}.$$

Thus the numerical discrepancy is at floating-point level.

Case	Mean	Std. dev.	L^1 vs GUE	KS vs GUE
Direct zero gaps	1.0000007062	0.4075236680	0.0430375588	0.0125223467
Cone quotient on zeros	1.0000007062	0.4075236680	0.0430375588	0.0125223467
Reduced $1 - \Re(\Delta G)$ on zeros	0.9999982317	0.4076656007	0.0428210425	0.0124312566
No-zero cone output	0.9999090173	0.4218851442	0.0003619897	0.0000853440
No-zero reduced driver	1.0000000000	0.4220158821	0.0001012558	0.0000015117

TABLE 1. Numerical metrics. The zero data reproduce the standard Montgomery–Odlyzko GUE comparison. The no-zero driver verifies the exact distribution-preserving character of the cone quotient when a GUE law is inserted algebraically.

6.4. **Summary statistics.**

6.5. **No-zero transducer test.** To isolate the algebra from the zeta data, the code also constructs a no-zero sequence. A deterministic golden-lattice quantile sample of the Wigner–GUE law is used as s_N. The model then builds G_N so that

$$1 - \Re(G_N - G_{N-1}) = s_N$$

up to a finite-sample centering and bounded rearrangement. With

$$Y_N = A_N + G_N$$

and H_N equal to the smooth RVM inverse height, the cone quotient output again follows the Wigner–GUE law. This is the numerical version of Theorem 3.2.

7. Interpretation

The cone quotient does not manufacture a new spacing law. It is stronger and simpler than that: it preserves the spacing law inserted into its recurrence. For arbitrary data satisfying

$$Y_N - e^{-i\alpha h_N} Y_{N-1} = 1,$$

the quotient returns h_N. For the zeta zeros this recurrence is an elementary consequence of

$$S_N - S_{N-1} = e^{i\alpha\gamma_N}.$$

Therefore the role of fitted variables is geometric and descriptive, not logical. The Wigner–GUE comparison belongs to the unfolded zero gaps themselves; the cone coordinates expose an exact algebraic channel through which those gaps are recovered.

8. Conclusion

The exact part of the result is the cone transducer theorem. It proves that the cone quotient returns any prescribed normalized spacing sequence exactly, including a Wigner–GUE sequence. The elementary zeta-zero identity proves that, when Y_N is constructed from the exponential walk of the zeros, the same quotient returns the actual zero gaps. The numerical section confirms this on 2,001,052 Odlyzko ordinates and records the GUE comparison in the cone coordinates.

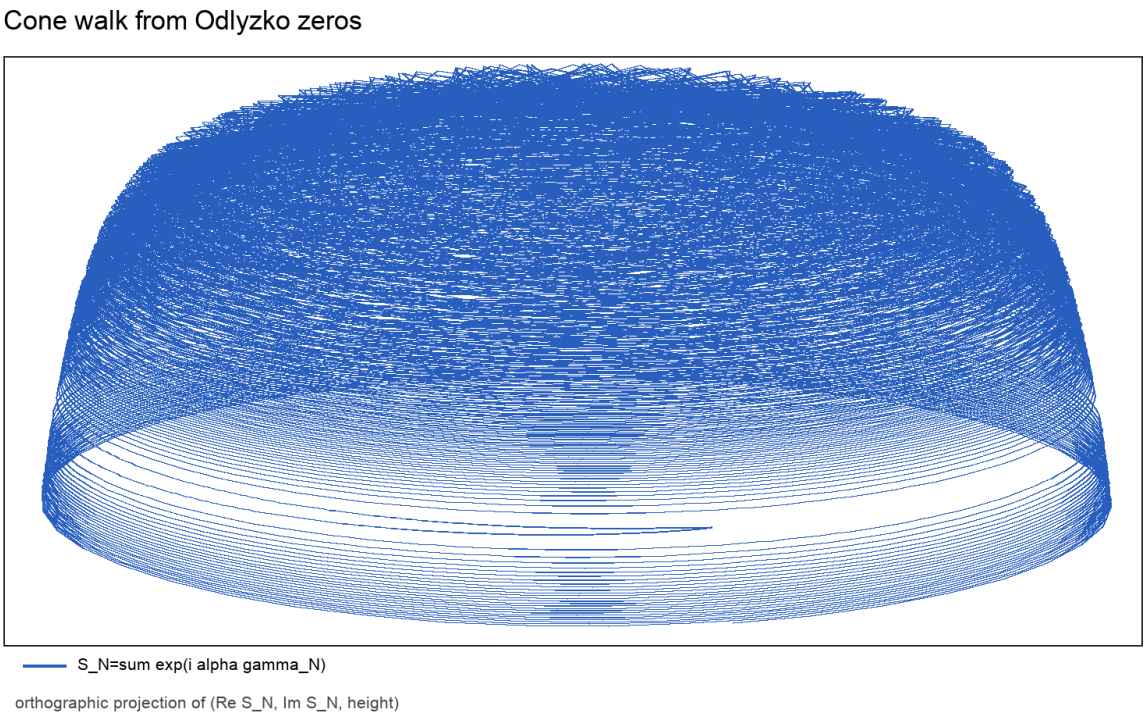


FIGURE 1. Orthographic projection of the cone walk $(\Re S_N, \Im S_N, \gamma_N)$ from the Odlyzko zero ordinates.

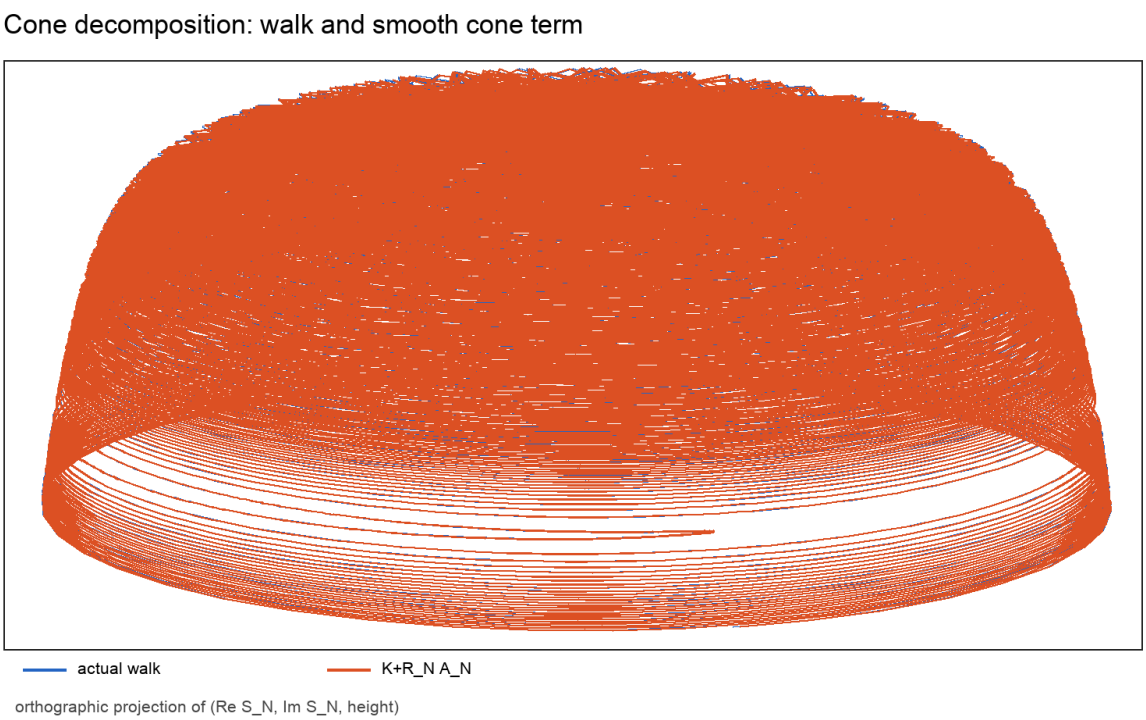


FIGURE 2. Cone decomposition: the observed walk and the smooth cone term $K + R_N A_N$. The gap extraction itself uses the exact quotient and does not depend on the smooth term being a complete predictor of the zeros.

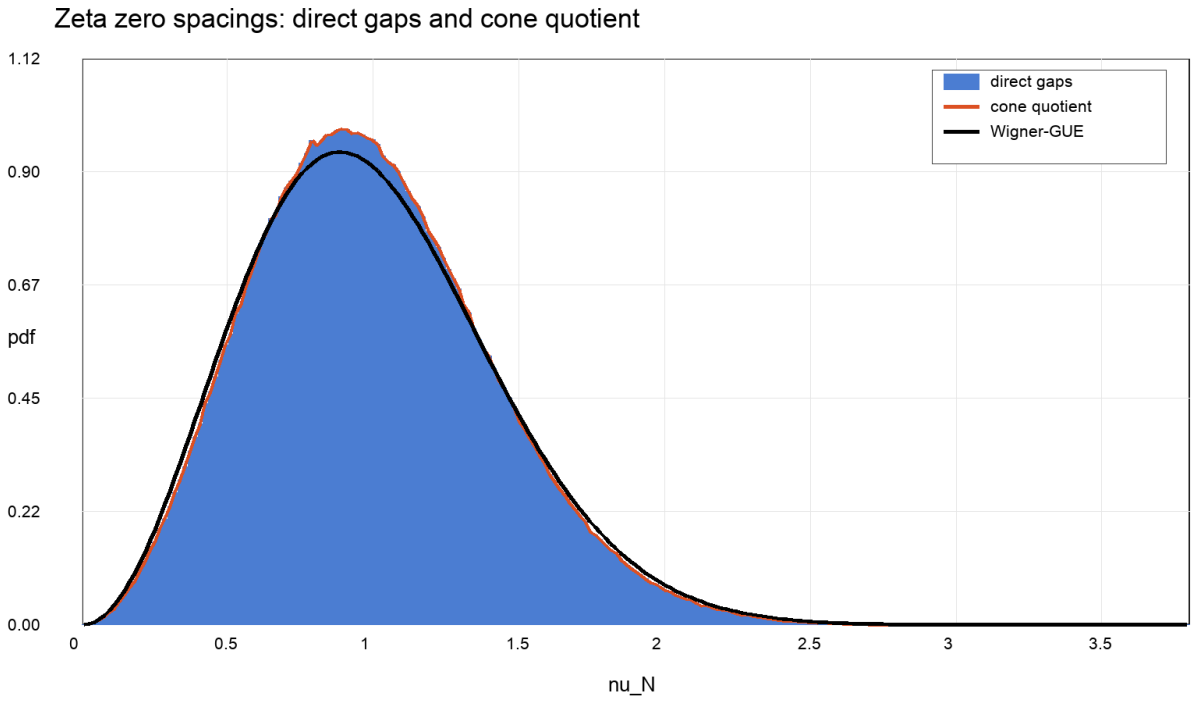


FIGURE 3. Direct unfolded zeta-zero gaps and cone-quotient gaps compared with the Wigner–GUE nearest-neighbour surmise. The two empirical curves coincide to plotting precision because the quotient is algebraically exact.

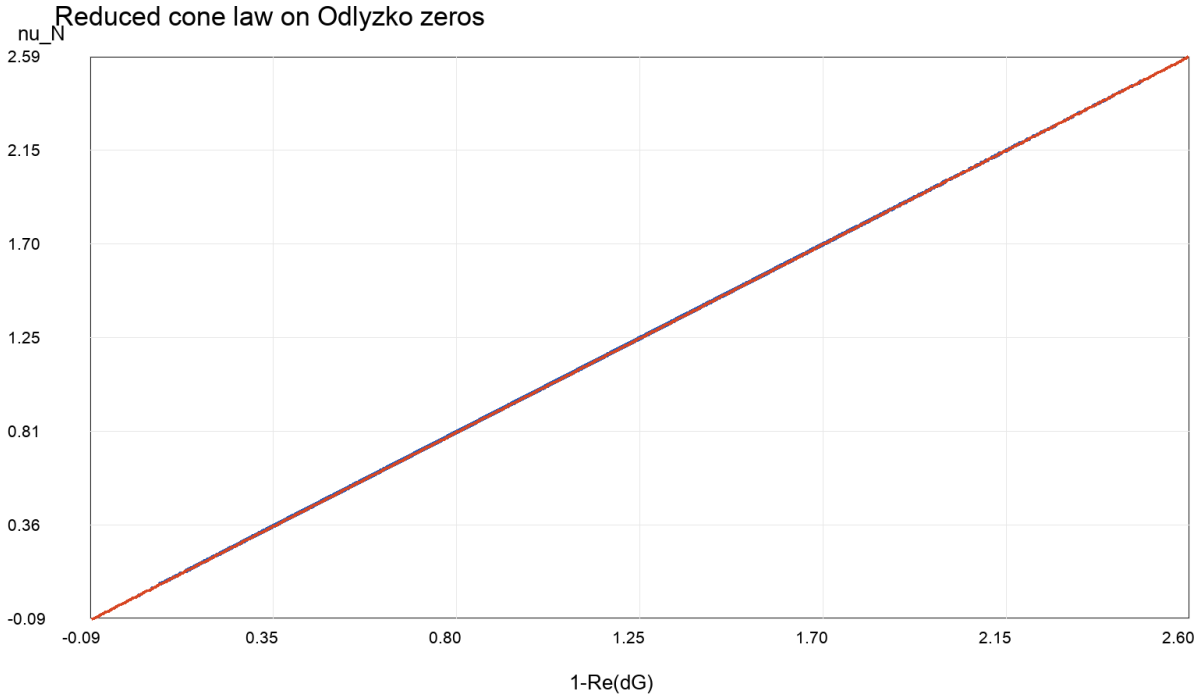


FIGURE 4. Reduced cone law on Odlyzko zeros: ν_N versus $1 - \Re(G_N - G_{N-1})$ on a downsampled set.

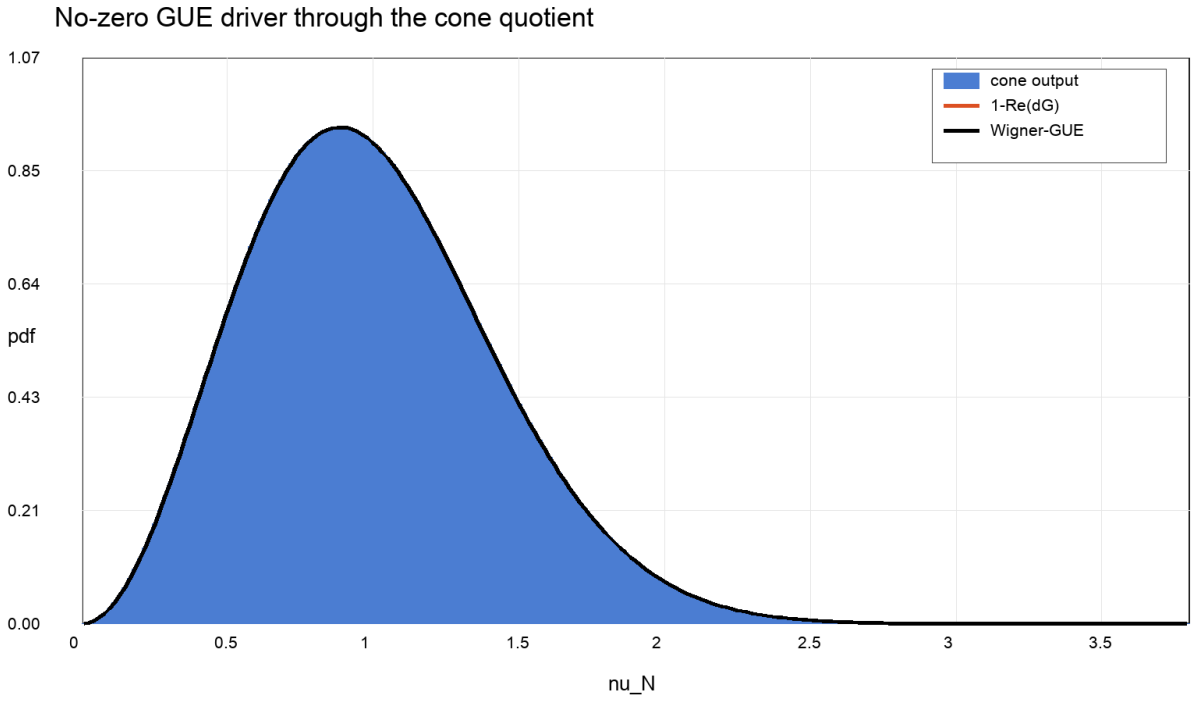


FIGURE 5. No-zero GUE driver through the cone quotient. No real zeros, `zeross.bin`, or `zLamb.bin` are used in this construction.

Appendix A. Python Code Used for All Figures and Tables

```
#!/usr/bin/env python3
"""Cone quotient algebra and GUE-spacing figures.

The script uses Python only. It reads zeross.bin for the empirical zeta-zero
checks, and it also constructs a no-zero GUE driver to verify the exact
transducer theorem numerically.
"""

from __future__ import annotations

import csv
import math
from pathlib import Path

import numpy as np
from PIL import Image, ImageDraw, ImageFont


ROOT = Path(__file__).resolve().parent.parent
OUT = ROOT / "CONE_GUE_ALGEBRA_ARTICLE"
ZEROS_FILE = ROOT / "zeross.bin"

NMAX_ZERO = None
NMODEL = 2_000_000

FIG_W = 1600
FIG_H = 950


def load_font(size: int) -> ImageFont.ImageFont:
    candidates = [
        "/System/Library/Fonts/Supplemental/Arial.ttf",
        "/System/Library/Fonts/Helvetica.ttc",
        "/Library/Fonts/Arial.ttf",
    ]
    for path in candidates:
        try:
            return ImageFont.truetype(path, size)
        except Exception:
            pass
    return ImageFont.load_default()


FONT_BIG = load_font(34)
FONT = load_font(26)
FONT_SMALL = load_font(21)


def erf_approx(x: np.ndarray) -> np.ndarray:
    sign = np.sign(x)
    ax = np.abs(x)
    p = 0.3275911
    a1 = 0.254829592
    a2 = -0.284496736
    a3 = 1.421413741
    a4 = -1.453152027
    a5 = 1.061405429
    t = 1.0 / (1.0 + p * ax)
    y = 1.0 - (((((a5 * t + a4) * t + a3) * t + a2) * t + a1) * t) * np.exp(-(ax * ax))
    return sign * y


def gue_pdf(s: np.ndarray) -> np.ndarray:
    y = np.zeros_like(s, dtype=np.float64)
    idx = s >= 0
    y[idx] = (32.0 / math.pi**2) * s[idx] ** 2 * np.exp(-4.0 * s[idx] ** 2 / math.pi)
    return y


def gue_cdf(s: np.ndarray) -> np.ndarray:
    y = np.zeros_like(s, dtype=np.float64)
    idx = s >= 0
    x = s[idx]
    y[idx] = erf_approx(2.0 * x / math.sqrt(math.pi)) - (4.0 * x / math.pi) * np.exp(
        -4.0 * x * x / math.pi
    )
    y[s > 20] = 1.0
    return y
```

```
def rvm_inverse_vector(n: np.ndarray) -> np.ndarray:
    two_pi = 2.0 * math.pi
    t = two_pi * (n + 1.0) / np.log(n + 10.0)
    t[t < 14.0] = 14.0
    for _ in range(30):
        f = t / two_pi * np.log(t / two_pi) - t / two_pi + 7.0 / 8.0
        df = (1.0 / two_pi) * np.log(t / two_pi)
        step = (f - n) / df
        step[~np.isfinite(step)] = 0.0
        t = t - step
        t[t <= two_pi] = two_pi + 1.0
    return t


def cone_fm_nu_from_y(y: np.ndarray, height: np.ndarray, alpha: float) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    q = (y[1:] - 1.0) / y[:-1]
    h = -np.angle(q) / alpha
    period = 2.0 * math.pi / alpha
    h = np.mod(h, period)
    h[h > period / 2.0] -= period
    h[h < 0.0] += period
    h0 = 2.0 * math.pi / np.log(height[1:] / (2.0 * math.pi))
    nu = h / h0
    eta = nu - 1.0
    return nu, eta, h


def compare_gue(x: np.ndarray, xmax: float = 3.8) -> dict[str, float]:
    x = x[np.isfinite(x)]
    edges = np.linspace(0.0, xmax, 220)
    centers = 0.5 * (edges[:-1] + edges[1:])
    dx = centers[1] - centers[0]
    hist, _ = np.histogram(x, bins=edges, density=True)
    pdf = gue_pdf(centers)
    xs = np.sort(x)
    emp = np.arange(1, xs.size + 1, dtype=np.float64) / xs.size
    ks = float(np.max(np.abs(emp - gue_cdf(xs))))
    return {
        "mean": float(np.mean(x)),
        "std": float(np.std(x, ddof=1)),
        "min": float(np.min(x)),
        "max": float(np.max(x)),
        "L1": float(np.sum(np.abs(hist - pdf)) * dx),
        "L2": float(math.sqrt(np.sum((hist - pdf) ** 2) * dx)),
        "KS": ks,
    }


def corr(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.corrcoef(a, b)[0, 1])


def polyline(draw: ImageDraw.ImageDraw, points: list[tuple[int, int]], color: tuple[int, int, int], width: int) -> None:
    if len(points) >= 2:
        draw.line(points, fill=color, width=width, joint="curve")


def project_3d(x: np.ndarray, y: np.ndarray, z: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    x = (x - np.mean(x)) / (np.std(x) + 1e-12)
    y = (y - np.mean(y)) / (np.std(y) + 1e-12)
    z = (z - np.min(z)) / (np.max(z) - np.min(z) + 1e-12)
    z = 3.0 * (z - 0.5)
    xr = 0.90 * x - 0.35 * y
    yr = 0.20 * x + 0.55 * y - 0.85 * z
    px = 130 + (xr - np.min(xr)) / (np.max(xr) - np.min(xr) + 1e-12) * (FIG_W - 230)
    py = 110 + (1.0 - (yr - np.min(yr)) / (np.max(yr) - np.min(yr) + 1e-12)) * (FIG_H - 230)
    return px, py


def draw_3d_path(
    path: Path,
    title: str,
    series: list[tuple[np.ndarray, np.ndarray, np.ndarray, str, tuple[int, int, int], int]],
) -> None:
    img = Image.new("RGB", (FIG_W, FIG_H), "white")
    draw = ImageDraw.Draw(img)
    draw.text((80, 35), title, fill=(0, 0, 0), font=FONT_BIG)
    draw.rectangle([80, 100, FIG_W - 60, FIG_H - 95], outline=(35, 35, 35), width=2)
    for x, y, z, label, color, width in series:
```

```
        px, py = project_3d(x, y, z)
        pts = [(int(a), int(b)) for a, b in zip(px, py)]
        polyline(draw, pts, color, width)
    lx = 105
    ly = FIG_H - 70
    for _, _, _, label, color, width in series:
        draw.line([lx, ly, lx + 55, ly], fill=color, width=max(4, width))
        draw.text((lx + 70, ly - 14), label, fill=(0, 0, 0), font=FONT_SMALL)
        lx += 390
    draw.text((95, FIG_H - 35), "orthographic projection of (Re S_N, Im S_N, height)", fill=(70, 70, 70), font=FONT_SMALL)
    img.save(path)


def draw_hist(path: Path, title: str, datasets: list[tuple[np.ndarray, str, tuple[int, int, int], str]], xmax: float = 3.8) ->
      None:
    img = Image.new("RGB", (FIG_W, FIG_H), "white")
    draw = ImageDraw.Draw(img)
    left, top, right, bottom = 115, 105, 65, 120
    pw = FIG_W - left - right
    ph = FIG_H - top - bottom
    edges = np.linspace(0.0, xmax, 220)
    centers = 0.5 * (edges[:-1] + edges[1:])
    hists = [np.histogram(data[np.isfinite(data)], bins=edges, density=True)[0] for data, _, _, _ in datasets]
    pdf = gue_pdf(centers)
    ymax = float(max([np.max(pdf)] + [np.max(h) for h in hists]) * 1.14)

    def xy(xv: float, yv: float) -> tuple[int, int]:
        return (
            left + int((xv / xmax) * pw),
            top + ph - int((yv / ymax) * ph),
        )

    draw.text((80, 35), title, fill=(0, 0, 0), font=FONT_BIG)
    draw.rectangle([left, top, left + pw, top + ph], outline=(35, 35, 35), width=2)
    for tick in np.arange(0.0, xmax + 1e-9, 0.5):
        x, _ = xy(float(tick), 0.0)
        draw.line([x, top, x, top + ph], fill=(230, 230, 230))
        draw.text((x - 16, top + ph + 12), f"{tick:g}", fill=(0, 0, 0), font=FONT_SMALL)
    for tick in np.linspace(0.0, ymax, 6):
        _, y = xy(0.0, float(tick))
        draw.line([left, y, left + pw, y], fill=(230, 230, 230))
        draw.text((20, y - 12), f"{tick:.2f}", fill=(0, 0, 0), font=FONT_SMALL)

    for hist, (_, label, color, mode) in zip(hists, datasets):
        if mode == "bars":
            for i, val in enumerate(hist):
                x0, y0 = xy(float(edges[i]), float(val))
                x1, y1 = xy(float(edges[i + 1]), 0.0)
                draw.rectangle([x0, y0, max(x0 + 1, x1), y1], fill=color)
        else:
            pts = [xy(float(c), float(v)) for c, v in zip(centers, hist)]
            polyline(draw, pts, color, 4)

    gue_pts = [xy(float(c), float(v)) for c, v in zip(centers, pdf)]
    polyline(draw, gue_pts, (0, 0, 0), 5)
    draw.text((left + pw // 2 - 35, FIG_H - 58), "nu_N", fill=(0, 0, 0), font=FONT)
    draw.text((20, top + ph // 2 - 20), "pdf", fill=(0, 0, 0), font=FONT)

    lx = left + pw - 315
    ly = top + 30
    draw.rectangle([lx - 15, ly - 15, lx + 285, ly + 105], fill=(255, 255, 255), outline=(70, 70, 70))
    for j, (_, label, color, mode) in enumerate(datasets + [(pdf, "Wigner-GUE", (0, 0, 0), "line")]):
        yy = ly + 32 * j
        if mode == "bars":
            draw.rectangle([lx, yy - 10, lx + 45, yy + 10], fill=color)
        else:
            draw.line([lx, yy, lx + 45, yy], fill=color, width=5)
        draw.text((lx + 60, yy - 13), label, fill=(0, 0, 0), font=FONT_SMALL)
    img.save(path)


def draw_scatter(path: Path, title: str, x: np.ndarray, y: np.ndarray, xlabel: str, ylabel: str) -> None:
    img = Image.new("RGB", (FIG_W, FIG_H), "white")
    draw = ImageDraw.Draw(img)
    left, top, right, bottom = 125, 105, 65, 125
    pw = FIG_W - left - right
    ph = FIG_H - top - bottom
    xlo, xhi = np.quantile(x, [0.001, 0.999])
    ylo, yhi = np.quantile(y, [0.001, 0.999])
    pad_x = 0.08 * (xhi - xlo)
    pad_y = 0.08 * (yhi - ylo)
```

```
    xlo -= pad_x
    xhi += pad_x
    ylo -= pad_y
    yhi += pad_y

    def xy(xv: float, yv: float) -> tuple[int, int]:
        return (
            left + int((xv - xlo) / (xhi - xlo) * pw),
            top + ph - int((yv - ylo) / (yhi - ylo) * ph),
        )

    draw.text((80, 35), title, fill=(0, 0, 0), font=FONT_BIG)
    draw.rectangle([left, top, left + pw, top + ph], outline=(35, 35, 35), width=2)
    for tick in np.linspace(xlo, xhi, 7):
        px, _ = xy(float(tick), ylo)
        draw.line([px, top, px, top + ph], fill=(232, 232, 232))
        draw.text((px - 35, top + ph + 12), f"{tick:.2f}", fill=(0, 0, 0), font=FONT_SMALL)
    for tick in np.linspace(ylo, yhi, 7):
        _, py = xy(xlo, float(tick))
        draw.line([left, py, left + pw, py], fill=(232, 232, 232))
        draw.text((20, py - 12), f"{tick:.2f}", fill=(0, 0, 0), font=FONT_SMALL)
    for xv, yv in zip(x, y):
        px, py = xy(float(xv), float(yv))
        if left <= px <= left + pw and top <= py <= top + ph:
            draw.ellipse([px - 2, py - 2, px + 2, py + 2], fill=(35, 90, 185))
    line_pts = [xy(float(t), float(t)) for t in np.linspace(max(xlo, ylo), min(xhi, yhi), 400)]
    polyline(draw, line_pts, (215, 70, 35), 4)
    draw.text((left + pw // 2 - 90, FIG_H - 58), xlabel, fill=(0, 0, 0), font=FONT)
    draw.text((left - 92, top - 44), ylabel, fill=(0, 0, 0), font=FONT)
    img.save(path)


def gue_quantile_lattice(m: int) -> np.ndarray:
    s_grid = np.linspace(0.0, 4.5, 300_000)
    f_grid = np.maximum.accumulate(gue_cdf(s_grid))
    keep = np.concatenate(([True], np.diff(f_grid) > 0.0))
    s_grid = s_grid[keep]
    f_grid = f_grid[keep]
    phi = (math.sqrt(5.0) - 1.0) / 2.0
    u = ((np.arange(m, dtype=np.float64) + 0.5) * phi) % 1.0
    u = np.clip(u, np.finfo(np.float64).eps, 1.0 - np.finfo(np.float64).eps)
    return np.interp(u, f_grid, s_grid)


def bounded_increment_rearrangement(d: np.ndarray) -> np.ndarray:
    pos = np.sort(d[d >= 0.0])[::-1]
    neg = np.sort(d[d < 0.0])
    out = np.empty_like(d)
    ip = 0
    ineg = 0
    x = 0.0
    for k in range(d.size):
        if x >= 0.0 and ineg < neg.size:
            val = neg[ineg]
            ineg += 1
        elif x < 0.0 and ip < pos.size:
            val = pos[ip]
            ip += 1
        elif ineg < neg.size:
            val = neg[ineg]
            ineg += 1
        else:
            val = pos[ip]
            ip += 1
        out[k] = val
        x += float(val)
    return out


def no_zero_gmodel(alpha: float, n_model: int) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    n = np.arange(1, n_model + 1, dtype=np.float64)
    height = rvm_inverse_vector(n)
    a = 0.5 + 1.0 / (2.0 * math.pi * alpha * alpha * height) - 1j / (2.0 * math.pi * alpha) * np.log(
        height / (2.0 * math.pi)
    )
    s = gue_quantile_lattice(n_model - 1)
    d_re = 1.0 - s
    d_re -= np.mean(d_re)
    d_re = bounded_increment_rearrangement(d_re)
    real_g = np.empty(n_model, dtype=np.float64)
    real_g[0] = 0.0
```

```
    real_g[1:] = np.cumsum(d_re)
    phi = (math.sqrt(5.0) - 1.0) / 2.0
    sigma_im = 0.010862192738467039
    phase = 2.0 * math.pi * ((np.arange(n_model, dtype=np.float64) * phi + 0.1732050807568877) % 1.0)
    g = real_g + 1j * sigma_im * np.sin(phase)
    y = a + g
    nu_cone, _, _ = cone_fm_nu_from_y(y, height, alpha)
    nu_red = 1.0 - np.real(np.diff(g))
    return nu_cone, nu_red, height


def write_summary(rows: list[dict[str, object]]) -> None:
    fields = ["case", "N", "mean", "std", "min", "max", "L1", "L2", "KS", "extra"]
    with (OUT / "CONE_GUE_ARTICLE_SUMMARY.csv").open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        for row in rows:
            writer.writerow(row)


def main() -> None:
    OUT.mkdir(exist_ok=True)
    circle_length = 3.0**math.pi * math.sqrt(math.exp(math.pi))
    alpha = 2.0 * math.pi / circle_length
    k_const = complex(-5.161990134502993, 2.9872973089668946)

    gamma = np.fromfile(ZEROS_FILE, dtype=np.float64)
    if NMAX_ZERO is not None:
        gamma = gamma[:NMAX_ZERO]
    gamma = gamma[np.isfinite(gamma)]
    n = gamma.size
    h = np.diff(gamma)
    h0 = 2.0 * math.pi / np.log(gamma[1:] / (2.0 * math.pi))
    nu_direct = h / h0
    eta_direct = nu_direct - 1.0

    r = np.exp(1j * alpha * gamma)
    s_walk = np.cumsum(r)
    a = 0.5 + 1.0 / (2.0 * math.pi * alpha * alpha * gamma) - 1j / (2.0 * math.pi * alpha) * np.log(
        gamma / (2.0 * math.pi)
    )
    y = np.exp(-1j * alpha * gamma) * (s_walk - k_const)
    g = y - a
    nu_cone, eta_cone, h_cone = cone_fm_nu_from_y(y, gamma, alpha)
    dreg_law = 1.0 - np.real(np.diff(g))

    step = max(1, n // 7000)
    idx = np.arange(0, n, step)
    idx_gap = np.arange(0, n - 1, step)
    smooth_s = k_const + r * a

    draw_3d_path(
        OUT / "FIG01_CONE_WALK_3D.png",
        "Cone walk from Odlyzko zeros",
        [
            (np.real(s_walk[idx]), np.imag(s_walk[idx]), gamma[idx], "S_N=sum exp(i alpha gamma_N)", (40, 95, 190), 2),
        ],
    )
    draw_3d_path(
        OUT / "FIG02_CONE_SMOOTH_FIT_3D.png",
        "Cone decomposition: walk and smooth cone term",
        [
            (np.real(s_walk[idx]), np.imag(s_walk[idx]), gamma[idx], "actual walk", (35, 100, 195), 2),
            (np.real(smooth_s[idx]), np.imag(smooth_s[idx]), gamma[idx], "K+R_N A_N", (220, 80, 35), 3),
        ],
    )
    draw_hist(
        OUT / "FIG03_ZEROS_DIRECT_CONE_GUE.png",
        "Zeta zero spacings: direct gaps and cone quotient",
        [
            (nu_direct, "direct gaps", (75, 125, 210), "bars"),
            (nu_cone, "cone quotient", (220, 80, 35), "line"),
        ],
    )
    draw_scatter(
        OUT / "FIG04_ZEROS_REDUCED_LAW.png",
        "Reduced cone law on Odlyzko zeros",
        dreg_law[idx_gap],
        nu_direct[idx_gap],
        "1-Re(dG)",
        "nu_N",
```

```
    )

    nu_model, nu_model_red, height_model = no_zero_gmodel(alpha, NMODEL)
    draw_hist(
        OUT / "FIG05_NO_ZERO_TRANSDUCER_GUE.png",
        "No-zero GUE driver through the cone quotient",
        [
            (nu_model, "cone output", (75, 125, 210), "bars"),
            (nu_model_red, "1-Re(dG)", (220, 80, 35), "line"),
        ],
    )

    metrics = []
    for case, arr, extra in [
        ("zeros_direct", nu_direct, ""),
        ("zeros_cone_quotient", nu_cone, f"max_abs_h_error={np.max(np.abs(h_cone - h)):.17g}"),
        ("zeros_reduced_1_minus_Re_dG", dreg_law, f"corr_with_nu={corr(dreg_law, nu_direct):.17g}"),
        ("no_zero_gmodel_cone", nu_model, f"corr_with_reduced={corr(nu_model, nu_model_red):.17g}"),
        ("no_zero_gmodel_reduced", nu_model_red, ""),
    ]:
        row = {"case": case, "N": arr.size, "extra": extra}
        row.update(compare_gue(arr))
        metrics.append(row)
    write_summary(metrics)

    report_lines = [
        "Cone GUE algebra article diagnostics",
        "====================================",
        f"zeross.bin values: {n}",
        f"gamma_1: {gamma[0]:.17g}",
        f"gamma_N: {gamma[-1]:.17g}",
        f"circleLength: {circle_length:.17g}",
        f"alpha: {alpha:.17g}",
        f"K: {k_const.real:.17g} {k_const.imag:+.17g}i",
        f"max |h_cone-h|: {np.max(np.abs(h_cone - h)):.17g}",
        f"mean |h_cone-h|: {np.mean(np.abs(h_cone - h)):.17g}",
        f"rms |h_cone-h|: {math.sqrt(np.mean((h_cone - h) ** 2)):.17g}",
        f"corr(nu, 1-Re(dG)): {corr(nu_direct, dreg_law):.17g}",
        f"mean |nu-(1-Re(dG))|: {np.mean(np.abs(nu_direct - dreg_law)):.17g}",
        f"rms |nu-(1-Re(dG))|: {math.sqrt(np.mean((nu_direct - dreg_law) ** 2)):.17g}",
        "",
    ]
    for row in metrics:
        report_lines.append(
            f"{row['case']}: mean={row['mean']:.17g}, std={row['std']:.17g}, "
            f"L1={row['L1']:.17g}, L2={row['L2']:.17g}, KS={row['KS']:.17g}, {row['extra']}"
        )
    (OUT / "CONE_GUE_ARTICLE_REPORT.txt").write_text("\n".join(report_lines) + "\n")
    print("\n".join(report_lines))


if __name__ == "__main__":
    main()
```